

Excessive Data Exposure — Interview Prep Notes

Note on OWASP mapping: API3:2019 "Excessive Data Exposure" was merged into **API3:2023 "Broken Object Property Level Authorization"** (combined with Mass Assignment) in the 2023 revision. Mention both if asked — shows currency.

1. Definition

- API returns full object/data in response, relying on **client-side filtering** to hide sensitive fields
- Server over-shares → client discards unwanted fields → attacker inspects raw response, bypasses client filter
- Root issue: **no server-side response filtering / schema enforcement**
- Distinct from BOLA (object-level access) — this is **property/field-level over-exposure within an authorized object**

2. OWASP / CWE

- OWASP API Top 10 2019: **API3:2019 – Excessive Data Exposure**
- OWASP API Top 10 2023: merged into **API3:2023 – Broken Object Property Level Authorization**
- CWE-213: Exposure of Sensitive Information Due to Incompatible Policy
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- CWE-359: Exposure of Private Personal Information

3. Common Variants

- Full DB object serialization (ORM `.toJSON()` / `SELECT *` dumped as-is)
- Verbose error messages leaking stack trace / internal paths / DB schema
- Debug/internal fields left in prod response (`is_admin`, `password_hash`, `internal_notes`)
- Nested object over-exposure (returning related user/child objects fully)
- GraphQL over-fetching (schema exposes fields not meant for that role)
- Metadata leakage (HTTP headers, API versioning info, server banners)

4. Attack Surface / Entry Points

- ☐ REST API response bodies (GET/POST/PUT responses)
- ☐ GraphQL query responses (introspection + object fields)
- ☐ Mobile app API endpoints (assume client filters — common mistake)
- ☐ Admin/internal APIs reused by public-facing frontend
- ☐ Error/exception responses (500s, validation errors)
- ☐ Search/autocomplete endpoints
- ☐ Batch/export endpoints
- ☐ WebSocket message payloads

5. Technical Testing Workflow

Step	Verify	Manual	Burp Suite	ZAP	Nuclei	Payload/Test
1. Map endpoints & responses	What fields API actually returns vs UI shows	Browse app, note fields in UI	Proxy + Site Map, review all responses	Spider + Passive scan	N/A (recon)	Compare UI-rendered fields vs raw JSON
2. Inspect raw responses	Extra fields beyond UI need	View raw response body	Repeater — resend request, inspect full JSON	Manual Request Editor	N/A	Check for PII, tokens, hashes, internal IDs
3. Diff role-based responses	Same endpoint, different roles return different sensitivity	Compare user vs admin token responses	Repeater w/ different session tokens	Same via Replacer/Scripts	N/A	Swap tokens, diff JSON keys
4. Force error states	Verbose errors leak schema/stack	Send malformed input	Repeater — fuzz params	Fuzzer add-on	Custom template: regex match <code>stacktrace Exception at com. at java.</code>	Invalid type, missing param, SQLi-like char
5. GraphQL introspection	Full schema exposure	<code>__schema{query}</code>	Repeater / BApp: GraphQL Raider	GraphQL add-on	Existing nuclei GraphQL introspection templates	<code>{__schema{types{name file ids{name}}}}</code>
6. Check nested/related objects	Over-fetch of linked entities	Inspect response for embedded objects	Repeater, expand nested JSON	Manual review	N/A	Request user → check if <code>address</code> , <code>payment_method</code> nested fully
7. Metadata/header leakage	Server/version disclosure	Check response headers	Proxy HTTP history	Passive scan alerts	Existing nuclei tech-detect/exposure templates	<code>Server</code> , <code>X-Powered-By</code> headers

- Nuclei: good for known exposure signatures (stack traces, debug endpoints, `.git`, swagger/openapi exposed), **not effective for logical over-exposure** — that's manual/Repeater-driven

6. Evidence to Collect

- Full raw response (request + response pair) showing sensitive field(s)
- Screenshot/annotation of UI (what's shown) vs raw JSON (what's returned)
- Field names + values redacted (e.g., `password_hash`, `ssn`, `internal_flag`)
- Role/token used to demonstrate no server-side filtering
- Repro steps (endpoint, method, headers, auth context)

7. Good PoC

- Request/response showing endpoint returns field X (e.g., `"national_id": "XXXXXX"`) not rendered in UI
- Demonstrate impact: field is sensitive (PII/credential/internal) + attacker-accessible via normal low-priv flow
- Ideally chain to show real harm — e.g., extract other users' PII via ID enumeration + excessive fields = mass data exposure

8. Attack Chains

- Excessive Data Exposure + **BOLA/IDOR** → enumerate IDs, harvest full PII at scale
- + **Broken Authentication** → exposed tokens/session data in response → session hijack
- + **Mass Assignment** → exposed field name reveals writable internal property → privilege escalation
- + **GraphQL introspection** → full schema map → targeted exploitation of internal fields
- Recon phase for further attacks (leaked internal IDs, emails, roles → targeted phishing/BFLA testing)

9. False Positives

- Field technically returned but non-sensitive (UI just chose not to render, no real impact)
- Data returned only to legitimately authorized role (verify RBAC context before flagging)
- Test/staging environment with dummy data
- Publicly available data duplicated in response (not sensitive in context)

10. Prerequisites for Exploitability

- Server returns raw/full object without field-level filtering
- Sensitive field present in response (PII, credentials, internal metadata)
- Attacker has (or can get) valid session/token to reach endpoint
- No response schema validation / allow-list enforcement at API gateway or app layer

11. Attacker Achieves

- Bulk PII harvesting (emails, phone, address, SSN/national ID)
- Credential/token/hash exposure → offline cracking or replay
- Internal schema/business logic disclosure → aids further attacks
- Competitive intel leakage (pricing, internal notes, business logic)

12. CIA Impact Table

Variant	C	I	A	Example CVSS Vector (3.1)
PII field leak (email, phone)	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Credential/hash/token leak	High	High*	None	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N (*via downstream account takeover)
Verbose error/stack trace	Medium	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Internal schema/GraphQL introspection	Low–Medium	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Financial data (account no., balance)	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

13. Typical CVSS

- Range: **4.3 (Medium)** – **7.5 (High)** depending on data sensitivity + auth requirement
- Base pattern: AV:N/AC:L/PR:L or N/UI:N/S:U/C:H/I:N/A:N
- Escalates to Critical if chained with IDOR/BOLA for mass enumeration

14. Business Impact (Financial App Context)

- Regulatory: **PCI-DSS** (cardholder data exposure → fines, loss of processing rights), **GDPR** (Art. 5/32 — data minimization, security of processing; breach notification within 72h), **RBI/local banking regs**, **GLBA** (US)
- Mass PII/financial data leak → regulatory fines, customer trust loss, mandatory breach disclosure
- Systemic risk: if pattern repeats across endpoints → indicates **no response-layer data governance**, high compounding risk (one enumeration bug + excessive exposure = mass breach)
- Reputational + potential class-action exposure in financial sector

15. Indicators / Signals

- Response JSON larger/richer than what UI renders
- Presence of `_id`, `internal_`, `debug`, `is_admin`, `hash`, `token` style keys in prod responses
- Verbose stack traces / DB error messages in responses
- GraphQL introspection enabled in production
- Swagger/OpenAPI spec publicly exposed showing full response schemas

16. Root Cause (Code Level)

- ORM serialization returns entire model:

```
return jsonify(user.__dict__) # or User.query.get(id).serialize()
```

- No DTO/response schema — API returns DB row 1:1
- GraphQL resolver returns full object without field-level auth
- Missing allow-list; relying on frontend to "hide" fields (security by obscurity)
- Root cause = **no explicit output contract**; server doesn't own what it exposes

17. Remediation (Secure Coding)

- **Explicit response DTOs / serializers** — allow-list fields, never return raw model

```
class UserResponseDTO:
    fields = ["id", "name", "email"] # explicit allow-list, no PII/internal fields
```

- Schema validation on output (e.g., JSON Schema, Pydantic `response_model` in FastAPI)
- GraphQL: field-level authorization directives, disable introspection in prod
- Generic error handling — no stack traces/internal messages to client
- Principle: **server decides what's exposed, never trust client to filter**

18. Compensating Controls

- API Gateway response filtering / transformation layer (strip fields at gateway if app can't be fixed immediately)
- WAF rules to block responses matching sensitive-data regex patterns
- Rate limiting + monitoring on endpoints with known over-exposure (limit bulk harvesting)
- Logging/alerting on large response payloads or high-frequency access to sensitive endpoints

19. Repeated Occurrence Indicates

- No centralized response schema/DTO pattern in SDLC
- Missing API design review / security requirements at design phase
- Absence of API-specific SAST/contract testing in CI/CD
- Weak "shift-left" culture — same as IDOR pattern, points to systemic API governance gap

20. Upstream Improvements

- SAST/Semgrep rule: flag `SELECT *`, `.toJSON()` / `.dict()` direct return without DTO wrapper, raw ORM object in `jsonify` / `res.json`
- Framework-level: enforce `response_model` (FastAPI), Serializer allow-lists (DRF), GraphQL shield/field-auth middleware
- OpenAPI schema contract testing in CI (response must match defined schema, extra fields fail build)
- Design checklist item: "Define response DTO before implementation" as part of API design review
- Disable GraphQL introspection + verbose errors via env config enforced at deploy pipeline

21. Sample Triage Response

"Confirmed: `GET /api/v1/user/{id}` returns additional fields (`ssn_last4`, `internal_risk_score`) not used/rendered by the client application, accessible by any authenticated low-privilege user. This constitutes Excessive Data Exposure (OWASP API3). Severity assessed as High due to PII sensitivity and low complexity of exploitation (no additional privilege required). Recommending immediate triage priority; requesting confirmation of data classification for `internal_risk_score` field to finalize severity."

22. Sample Developer Remediation Note

"Endpoint `/api/v1/user/{id}` currently serializes the full User ORM object. Replace with explicit response DTO returning only `id`, `name`, `email` fields required by frontend. Remove `ssn_last4`, `password_hash`, `internal_risk_score` from serialization. Add response schema validation (Pydantic/DRF serializer) to prevent future field creep. Retest: confirm response payload via Burp Repeater matches allow-listed schema post-fix."

23. Real-World Example

- **Venmo (2019)**: public API returned full transaction data (payer, payee, note) even for accounts with "private" setting on UI — client-side privacy filter, server exposed everything; researcher scraped millions of transactions
- **USPS Informed Delivery API (2018)**: authenticated API exposed data belonging to other users (account info) due to excessive/improper data return
- Common bug bounty pattern: mobile app APIs returning full user object (including `is_premium`, `internal_id`, `email_verified_token`) meant only for internal app logic, discoverable via proxying mobile traffic